NodiBondhu: Team Instructions

Challenge: NESFIC-D-18, flood digital twin. Pilot: **Simaluguri, Sivasagar district, Assam**. Deadline: **12:00 midnight tonight**.

Where things stand

A first version of the website runs on Person 1's laptop: map, three data layers, flood slider, settlement counts, dashboard tiles and a demo alert log.

Part	State
Rain forecast	Live from Open-Meteo
Flood scenarios (+1 to +6 m)	Working, but **not yet checked against a real flood and probably too large**. The model floods the Simaluguri point only from +6 m
River level	**Missing**: needs a real CWC reading
Past flood from satellite	**Missing**: needed to check the model
Settlement counts	Too low: only 70 places are mapped in this area
Alerts	Demo log only, nothing is sent

The biggest job now is real evidence. It matters more than new features.

Rules for everyone

- Never enter a number without a real source. AI tools can invent figures, so check each one on the original site.
- Never say the model is accurate until it has been compared with a real flood.
- Keep every API key out of the repo and out of shared documents.
- Check in at **3 pm, 6 pm and 8 pm**. Features freeze at **9:45 pm**. Submit by **11:45 pm**.

Person 1: main laptop with Claude

Start the app. Run these two commands from `D:\Hackathon\NodiBondhu`, each in its own terminal:

```
backend\.venv\Scripts\python -m uvicorn main:app --app-dir backend --port 8000
npm --prefix frontend run dev
```

Then open http://localhost:5173.

Steps

1. Create a shared Drive folder and a shared Google Sheet, and send both links to the team.
2. When a file arrives from Person 2 or 3, save it to the project and tell Claude. Claude adds it to the app.
3. Give Claude the first CWC reading from the shared sheet, and it updates the river level tile.
4. Give Claude the past flood file. Claude compares it with the scenarios and adjusts the model.
5. By 9:45 pm: create Vercel and Render accounts, then deploy with Claude's help.
6. At 10:45 pm: record the demo with Person 3's narration.
7. At 11:15 pm: run the checklist in `PLAN.md` and submit on the NESFIC portal.

Person 2: laptop with Gemini Pro

You now own the evidence. The terrain and first flood model are done, so your job is to check them.

Step 1: Agree the flood to check (with Person 3, by 1:30 pm). Pick one recent flood affecting Simaluguri or Sivasagar district, with a known date from ASDMA or news reports. It should show whether Simaluguri itself flooded, because that tests the +6 m result.

Step 2: Get the real flood extent from satellite (by 3 pm).

1. Register at https://console.cloud.google.com/earth-engine for noncommercial use. If approval stalls, use Bhuvan's flood maps instead.

2. Open the Earth Engine Code Editor at https://code.earthengine.google.com.
3. Paste this prompt into Gemini, filling in the dates:

> Write a Google Earth Engine JavaScript script for the Code Editor. Area: rectangle with west 94.40, south 26.65, east 95.00, north 27.20 in Assam, India. Use COPERNICUS/S1_GRD, VV polarisation, IW mode, one orbit direction. Compare a dry-season median (BEFORE DATES) with a flood-period median (FLOOD DATES) and map new water with a threshold on the difference in dB. Remove permanent water using JRC/GSW1_4/GlobalSurfaceWater, remove slopes above 5 degrees using a DEM, and remove small speckles. Show the result on the map and export it as a GeoJSON polygon file to Google Drive, using a 30 to 100 m scale. Explain which values I should adjust.

1. Compare the result with reports of that flood. Adjust the threshold if it looks wrong, and note the value you used.
2. Save it as `flood_extent_YYYY-MM-DD.geojson` in the shared Drive folder, and add a note with the image dates, threshold and source.

Step 3: Find better village data (by 5 pm). Look on Bhuvan, the Assam GIS geoportal, NESDR, data.gov.in and Census sources for villages with population. Share the file with its source and licence, or tell the team if you can't find one.

Step 4: Find embankment locations (by 6 pm). Look for maps of embankments around Simaluguri, along the Dikhow, Disang and Brahmaputra. The model ignores them, and that's its biggest weakness. Even a rough map with a source helps.

Step 5: Write proposal notes (by 8 pm). Send Person 3 short notes for sections 7 (data sources and access) and 9 (test method and results).

Step 6: Test (8 to 9:45 pm). Try to break the site, and check every number against its source.

Person 3: tablet

You own the proposal and the real-world facts. Use Google Docs and Sheets.

Step 1: River readings (by 2 pm).

1. Go to https://ffs.india-water.gov.in and open **Table View**.
2. Find the gauge closest to Simaluguri on the Dikhow, the Disang or the Brahmaputra. Note its name, as the app does not have one yet.
3. In the shared sheet, record: station, river, water level (m), warning level (m), danger level (m), date and time, and the page link.
4. If no reading is available, write that down and tell Person 1. Don't estimate one.

Step 2: Flood reports (by 3 pm). With Person 2, choose the past flood. From https://asdma.assam.gov.in, collect the date, affected places and figures, with links.

Step 3: Start the proposal (by 2 pm). Create the document with these 12 headings, as required by `uspr.pdf`:

1. Problem understanding and user workflow
2. Proposed solution and key features
3. User journey, wireframes and prototype
4. Technology and solution architecture
5. Development methodology and intermediate steps
6. Development timeline and milestones
7. Data and information requirements
8. Team, manpower and expertise
9. Testing, validation and evidence
10. Security, privacy, safety and compliance
11. Innovation and differentiation
12. Resources, risks and pilot validation

Step 4: Write the sections, in this order.

By	Sections	Who helps
3 pm	1, 2	Use the challenge page
6 pm	3, 5, 6, 8	Ask Person 1 for screenshots

By	Sections	Who helps
8 pm	10, 11, 12	Search for FLEWS, the existing Assam flood warning system by NESAC and ASDMA, for section 11
8 pm	4, 7, 9	Person 1 and Claude for 4; Person 2's notes for 7 and 9

What to say in the proposal

- Label the current work as a prototype, and say which data is live, sample or simulated.
- Section 7: do not assume access to government data you haven't received.
- Section 9: report the real comparison with the past flood, even if the match is weak, and explain how you'd improve it.
- Section 10: an officer must approve every alert before it goes out. Claude can't confirm any legal or licence claim for you, so check Open-Meteo and Earth Engine terms for commercial use.
- Section 12: list possible paid licences as a cost, and describe a pilot in one district.

Step 5: Demo (by 10:45 pm). Test the site on your tablet, then write and narrate a script of 2 to 3 minutes: the problem, the three data types, the flood slider, the alert approval, then the limits and next steps.

Step 6: Finish (by 10:45 pm). Add screenshots, export the proposal as a PDF and send it to Person 1.

What Claude does next on Person 1's laptop

- Adds real CWC readings to the app
- Adds the past flood as an Observed layer and measures how well the model matches it
- Improves the model using that result and any embankment data
- Replaces settlement counts if better village data arrives
- Helps draft sections 4, 7 and 9, and helps with deployment